PRACTICAL 1- Simulating Digital Evidence Collection

Write a Python script to simulate collecting basic digital evidence from system logs and files (e.g., fetching system log files).

```
import os
import shutil
import hashlib
import csv
from datetime import datetime

source_folder = "Evidence_Source"
destination_folder = "Collected_Evidence"

os.makedirs(destination_folder, exist_ok=True)

report_file = "evidence_report.csv"



def calculate_hash(file_path):
    sha256 = hashlib.sha256()

    with open(file_path, "rb") as file:
        while True:
            data = file.read(4096)

            if not data:
                break

            sha256.update(data)
-
```

```
    return sha256.hexdigest()


with open(report_file, "w", newline="") as csvfile:

    writer = csv.writer(csvfile)

    writer.writerow([
        "File Name",
        "File Size",
        "Modified Time",
        "SHA-256 Hash"
    ])

    for file_name in os.listdir(source_folder):

        source_path = os.path.join(source_folder, file_name)

        if os.path.isfile(source_path):

            destination_path = os.path.join(
                destination_folder,
                file_name
            )

            shutil.copy2(source_path, destination_path)

            file_size = os.path.getsize(source_path)

            modified_time = datetime.fromtimestamp(
```

```
            os.path.getmtime(source_path)
        )

        file_hash = calculate_hash(source_path)

        writer.writerow([
            file_name,
            file_size,
            modified_time,
            file_hash
        ])

        print("Collected:", file_name)


print("\nDigital evidence collection completed.")
print("Evidence saved in:", destination_folder)
print("Report saved as:", report_file)
```

TEXT DOCUMENT

system_log.txt

2026-08-21 09:00:01 - User login successful

2026-08-21 09:15:22 - File accessed

2026-08-21 09:30:45 - USB device connected

2026-08-21 10:05:12 - User logout

Suspicious_file.txt

This is a sample file for digital forensic investigation.

PRACTICAL 2 — Parsing Metadata from Files

Develop a script to extract and display metadata (author, creation date, file type) from image, PDF, or text files.

```
import os
import mimetypes
from datetime import datetime
from PIL import Image
from pypdf import PdfReader
from tkinter import Tk, filedialog



def get_file_type(file_path):
    file_type, _ = mimetypes.guess_type(file_path)

    if file_type:
        return file_type

    return "Unknown"



def get_creation_date(file_path):
    timestamp = os.path.getctime(file_path)
    return datetime.fromtimestamp(timestamp).strftime("%Y-%m-%d %H:%M:%S")



def get_image_metadata(file_path):
    image = Image.open(file_path)

    author = "Not available"
```

```
    creation_date = "Not available"

    exif_data = image.getexif()

    if exif_data:
        for tag_id, value in exif_data.items():

            if tag_id == 315:
                author = value

            if tag_id == 36867:
                creation_date = value

    return author, creation_date



def get_pdf_metadata(file_path):
    reader = PdfReader(file_path)

    metadata = reader.metadata

    author = "Not available"
    creation_date = "Not available"

    if metadata:
        if metadata.author:
            author = metadata.author

        if metadata.creation_date:
            creation_date = str(metadata.creation_date)

    return author, creation_date
```

```
def get_text_metadata(file_path):
    author = "Not available"
    creation_date = get_creation_date(file_path)

    return author, creation_date



def main():

    root = Tk()
    root.withdraw()

    file_path = filedialog.askopenfilename(
        title="Select a file",
        filetypes=[
            ("All Files", "*.*"),
            ("Image Files", "*.jpg *.jpeg *.png"),
            ("PDF Files", "*.pdf"),
            ("Text Files", "*.txt")
        ]
    )

    if not file_path:
        print("No file selected.")
        return

    file_name = os.path.basename(file_path)
    file_type = get_file_type(file_path)

    extension = os.path.splitext(file_path)[1].lower()
```

```
    if extension in [".jpg", ".jpeg", ".png"]:
        author, creation_date = get_image_metadata(file_path)

    elif extension == ".pdf":
        author, creation_date = get_pdf_metadata(file_path)

    elif extension == ".txt":
        author, creation_date = get_text_metadata(file_path)

    else:
        author = "Not available"
        creation_date = get_creation_date(file_path)

    print("\n========== FILE METADATA ==========")
    print("File Name     :", file_name)
    print("File Type     :", file_type)
    print("Author        :", author)
    print("Creation Date :", creation_date)
    print("File Location :", file_path)
    print("====================================\n")


main()
```

PRACTICAL 3

HASHING FILES FOR INTEGERTY CHECK

Create a program to generate and compare file hashes using algorithms like **MD5 and SHA-256**

```
import hashlib
filename = input("Enter the file name: ")


try:
    with open(filename, "rb") as file:
        data = file.read()


    md5_hash = hashlib.md5(data).hexdigest()


    sha256_hash = hashlib.sha256(data).hexdigest()
    print("\nMD5 Hash:")
    print(md5_hash)


    print("\nSHA-256 Hash:")
    print(sha256_hash)


except FileNotFoundError:
    print("File not found. Please check the file name.")
```

TEXT DOCUMENT

evidence.txt

This is my digital evidence file

Modified txt :

This is my modified digital evidence file

ACTUAL COMPARISON

```
import hashlib

filename = input("Enter the file name: ")

try:
    with open(filename, "rb") as file:
        data = file.read()

    md5_hash = hashlib.md5(data).hexdigest()
    sha256_hash = hashlib.sha256(data).hexdigest()

    print("\nCurrent MD5:")
    print(md5_hash)

    print("\nCurrent SHA-256:")
    print(sha256_hash)
```

```
    original_hash = input("\nEnter the original SHA-256 hash: ").strip()

    if sha256_hash == original_hash:
        print("\nRESULT: File is unchanged.")
        print("Integrity check PASSED.")
    else:
        print("\nRESULT: File has been modified.")
        print("Integrity check FAILED.")

except FileNotFoundError:
    print("File not found. Please check the file name.")
```

PRACTICAL 4: SIMULATING FILE RECOVERY

Objective: Write a Python program to recover simulated deleted text files from a directory by scanning the filesystem.

```
import os
import shutil
import hashlib
import csv
from datetime import datetime

SOURCE_DIR = "Evidence_Source"
RECOVERY_DIR = "Recovered_Files"
REPORT_FILE = "recovery_report.csv"

os.makedirs(RECOVERY_DIR, exist_ok=True)

def calculate_hash(file_path):
    sha256 = hashlib.sha256()

    with open(file_path, "rb") as file:
        while True:
            data = file.read(4096)

            if not data:
                break
```

```
            sha256.update(data)

    return sha256.hexdigest()

def is_text_file(file_path):
    try:
        with open(file_path, "r", encoding="utf-8") as file:
            file.read()

        return True

    except (UnicodeDecodeError, OSError):
        return False

def recover_deleted_files():

    recovery_records = []
    recovered_count = 0

    print("\nStarting file recovery scan...")
    print("-" * 45)

    for root, folders, files in os.walk(SOURCE_DIR):
```

```
for filename in files:

    if filename.endswith(".deleted"):

        original_path = os.path.join(root, filename)

        print("\nFound:", original_path)

        if is_text_file(original_path):

            recovered_name = filename.replace(
                ".deleted", ""
            )

            recovered_path = os.path.join(
                RECOVERY_DIR, recovered_name
            )

            shutil.copy2(
                original_path,
                recovered_path
            )
```

```
        file_hash = calculate_hash(
            recovered_path
        )

        recovery_records.append([
            filename,
            recovered_name,
            os.path.getsize(recovered_path),
            file_hash,
            datetime.now().strftime(
                "%Y-%m-%d %H:%M:%S"
            ),
            "Recovered"
        ])

        recovered_count += 1

        print("Status: Recovered")
        print("Saved as:", recovered_name)
        print("SHA-256:", file_hash)

    else:
        print("Status: Not a readable text file")
```

```
with open(
    REPORT_FILE,
    "w",
    newline="",
    encoding="utf-8"
) as report:

    writer = csv.writer(report)

    writer.writerow([
        "Original File",
        "Recovered File",
        "Size (Bytes)",
        "SHA-256 Hash",
        "Recovery Time",
        "Status"
    ])

    writer.writerows(recovery_records)

print("\n" + "-" * 45)
print("Recovery process completed.")
print("Total files recovered:", recovered_count)
print("Report saved as:", REPORT_FILE)
```

```
if __name__ == "__main__":
    recover_deleted_files()
```

TEXT DOCUMENT

notes.txt.deleted - This is a simulated deleted file. Digital forensics practical.

Evidence.txt.deleted - Evidence ID: DF001 This file is used for recovery testing

PRACTICAL 5: FILE TYPE IDENTIFICATION USING MAGIC NUMBERS

Develop a Python script that identifies file types using magic numbers (file signatures) instead of relying on file extensions

```
import os

MAGIC_NUMBERS = {
    "JPEG Image": [b"\xFF\xD8\xFF"],
    "PDF Document": [b"%PDF"]
}


def identify_file_type(file_path):

    if not os.path.isfile(file_path):
        print("Error: File does not exist.")
        return

    try:
        # Open the file in binary read mode
        with open(file_path, "rb") as file:
```

```
    header = file.read(16)

print("\nFile Analysis Report")
print("-" * 35)
print("File Name:", os.path.basename(file_path))

print("Header Bytes:", header.hex(" ").upper())

detected_type = "Unknown File Type"

for file_type, signatures in MAGIC_NUMBERS.items():

    for signature in signatures:

        if header.startswith(signature):
            detected_type = file_type
            break

    if detected_type != "Unknown File Type":
        break

print("Identified Type:", detected_type)

extension = os.path.splitext(file_path)[1]
```

```
        print("File Extension:",
              extension if extension else "None")

        if detected_type == "Unknown File Type":
            print("Result: Signature not found.")
        else:
            print("Result: File signature identified.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as error:
        print("Error while reading file:", error)


print("MAGIC NUMBER FILE TYPE DETECTOR")
print("=" * 40)

file_path = input("Enter the path of the file: ")

identify_file_type(file_path)
```

PRACTICAL 6: CARVING FILES FROM RAW DISK IMAGES

To implement a Python program to extract JPEG files from a raw binary file using file signatures.

File_carving.py

```
import os
import csv
import hashlib
from datetime import datetime

# File Carving: Recover JPEG Files from Raw Binary Data

JPEG_START = b"\xFF\xD8\xFF"
JPEG_END = b"\xFF\xD9"



def calculate_hash(data):
    """Calculate SHA-256 hash of recovered data."""
    return hashlib.sha256(data).hexdigest()



def read_raw_file(filename):
    """Read the raw binary file."""
    try:
```

```
        with open(filename, "rb") as file:
            data = file.read()

        print("\nRaw file loaded successfully.")
        print("Total bytes:", len(data))
        return data

    except FileNotFoundError:
        print("Error: Raw binary file not found.")
        return None

    except OSError as error:
        print("Error reading file:", error)
        return None



def carve_jpegs(data, output_folder):
    """Find and extract JPEG files from binary data."""

    os.makedirs(output_folder, exist_ok=True)

    recovered_count = 0
    search_position = 0
    recovery_records = []
```

```
while True:
    # Find the beginning of a JPEG image
    start = data.find(JPEG_START, search_position)

    if start == -1:
        break

    # Find the JPEG ending marker
    end_marker = data.find(JPEG_END, start + len(JPEG_START))

    if end_marker == -1:
        print("Incomplete JPEG found at offset:", start)
        search_position = start + len(JPEG_START)
        continue

    # Include the ending marker
    end = end_marker + len(JPEG_END)

    # Extract image bytes
    image_data = data[start:end]

    # Check minimum length
    if len(image_data) < 10:
```

```
    search_position = start + len(JPEG_START)
    continue

recovered_count += 1

# Create output filename
output_name = f"recovered_{recovered_count}.jpg"
output_path = os.path.join(output_folder, output_name)

# Save recovered image
with open(output_path, "wb") as output_file:
    output_file.write(image_data)

# Calculate image hash
file_hash = calculate_hash(image_data)

print("\nJPEG image recovered!")
print("Filename:", output_name)
print("Start offset:", start)
print("End offset:", end - 1)
print("File size:", len(image_data), "bytes")
print("SHA-256:", file_hash)

recovery_records.append({
```

```
            "filename": output_name,
            "start_offset": start,
            "end_offset": end - 1,
            "size_bytes": len(image_data),
            "sha256": file_hash,
            "status": "Extracted"
        })

        # Continue searching after the recovered image
        search_position = end

    return recovery_records


def generate_report(records, output_folder):
    """Save recovery details in a CSV report."""

    report_path = os.path.join(
        output_folder, "recovery_report.csv"
    )

    fieldnames = [
        "filename",
        "start_offset",
```

```
        "end_offset",
        "size_bytes",
        "sha256",
        "status"
    ]

    with open(report_path, "w", newline="") as report_file:
        writer = csv.DictWriter(
            report_file, fieldnames=fieldnames
        )

        writer.writeheader()
        writer.writerows(records)

    print("\nRecovery report saved to:", report_path)


def main():
    print("=" * 50)
    print("       JPEG FILE CARVING TOOL")
    print("=" * 50)

    raw_filename = input(
        "\nEnter raw binary filename (raw_image.bin): "
```

```
).strip()

if not raw_filename:
    raw_filename = "raw_image.bin"

output_folder = "recovered_files"

# Read raw binary data
data = read_raw_file(raw_filename)

if data is None:
    return

print("\nSearching for JPEG signatures...")

# Extract JPEG images
records = carve_jpegs(data, output_folder)

# Generate CSV report
generate_report(records, output_folder)

print("\n" + "=" * 50)
print("            RECOVERY SUMMARY")
print("=" * 50)
```

```
    print("Total JPEG files recovered:", len(records))
    print("Output folder:", output_folder)
    print("Completed at:", datetime.now().strftime(
        "%Y-%m-%d %H:%M:%S"
    ))

    if records:
        print("Result: JPEG extraction completed.")
    else:
        print("Result: No complete JPEG markers found.")

    print("=" * 50)


if __name__ == "__main__":
    main()
```

prepare_raw.py

```
with open("sample.jpg", "rb") as file:
    image = file.read()

raw_data = b"\x00\x11\x22\x33" * 100 + image
```

```
raw_data += b"\x44\x55\x66\x77" * 100

with open("raw_image.bin", "wb") as file:
    file.write(raw_data)

print("Raw binary file created successfully.")
```

PRACTICAL 7: CRYPTOCURRENCY WALLET FORENSICS USING PYTHON

To develop a Python script to analyze Bitcoin wallet transaction data and calculate transaction totals and net balance changes.

Wallet_data.json


```
{
  "wallet_address": "WALLET_A",
  "transactions": [
    {
      "txid": "TX001",
      "from": "EXTERNAL_A",
      "to": "WALLET_A",
      "amount_btc": 0.50000000,
      "fee_btc": 0.00000000,
      "date": "2026-10-01"
    },
    {
      "txid": "TX002",
      "from": "WALLET_A",
      "to": "EXTERNAL_B",
      "amount_btc": 0.10000000,
      "fee_btc": 0.00001000,
      "date": "2026-10-03"
```

```
    },
    {
      "txid": "TX003",
      "from": "EXTERNAL_C",
      "to": "WALLET_A",
      "amount_btc": 0.25000000,
      "fee_btc": 0.00000000,
      "date": "2026-10-05"
    },
    {
      "txid": "TX004",
      "from": "WALLET_A",
      "to": "EXTERNAL_D",
      "amount_btc": 0.05000000,
      "fee_btc": 0.00002000,
      "date": "2026-10-07"
    }
  ]
}
```

Wallet_forensics.py

```
import json
import csv
```

```
import os
from decimal import Decimal, InvalidOperation
from collections import Counter
from datetime import datetime



def load_wallet_data(filename):
    """Load wallet data from a JSON file."""

    try:
        with open(filename, "r", encoding="utf-8") as file:
            data = json.load(file)

        if not isinstance(data, dict):
            raise ValueError("Wallet data must be a JSON object.")

        if not isinstance(data.get("transactions"), list):
            raise ValueError("Transactions must be a list.")

        if not isinstance(data.get("wallet_address"), str):
            raise ValueError("Wallet address is missing.")

        print("\nWallet data loaded successfully.")
        return data
```

```
    except FileNotFoundError:
        print("Error: Wallet data file not found.")
    except json.JSONDecodeError:
        print("Error: Invalid JSON format.")
    except (OSError, ValueError) as error:
        print("Error:", error)

    return None


def classify_transaction(transaction, wallet_address):
    """Classify a transaction as incoming, outgoing or unrelated."""

    sender = transaction.get("from")
    receiver = transaction.get("to")

    if sender == wallet_address and receiver == wallet_address:
        return "Self-transfer"

    if receiver == wallet_address:
        return "Incoming"

    if sender == wallet_address:
```

```
        return "Outgoing"

    return "Unrelated"


def analyze_transactions(data):
    """Analyze transactions and calculate the net balance."""

    wallet_address = data["wallet_address"]
    transactions = data["transactions"]

    incoming_total = Decimal("0")
    outgoing_total = Decimal("0")
    fee_total = Decimal("0")

    records = []
    transaction_count = Counter()

    print("\n" + "=" * 65)
    print("             BITCOIN WALLET FORENSICS")
    print("=" * 65)

    print("Wallet identifier:", wallet_address)
    print("Total transaction records:", len(transactions))
```

```
for number, transaction in enumerate(transactions, start=1):

    try:
        txid = str(transaction["txid"])
        sender = str(transaction["from"])
        receiver = str(transaction["to"])
        date = str(transaction["date"])

        amount = Decimal(str(transaction["amount_btc"]))
        fee = Decimal(str(transaction.get("fee_btc", 0)))

        if not amount.is_finite() or not fee.is_finite():
            raise ValueError("Amount and fee must be finite.")

        if amount < 0 or fee < 0:
            raise ValueError("Negative amounts or fees are invalid.")

        if not txid.strip():
            raise ValueError("Transaction ID is empty.")

        if not sender.strip() or not receiver.strip():
            raise ValueError("Sender or receiver is empty.")
```

```
except (KeyError, ValueError, InvalidOperation,
        TypeError) as error:
    print("Skipping invalid transaction:", error)
    continue

category = classify_transaction(
    transaction, wallet_address
)

# Calculate the wallet's net change
if category == "Incoming":
    incoming_total += amount

elif category == "Outgoing":
    outgoing_total += amount

elif category == "Self-transfer":
    # The transfer itself does not change the balance.
    pass

transaction_fee = Decimal("0")

# Count fees only for outgoing or self-transfers
# explicitly assumed to be paid by this wallet.
```

```
if category in ("Outgoing", "Self-transfer"):
    transaction_fee = fee
    fee_total += fee

transaction_count[category] += 1

print("\nTransaction:", number)
print("Transaction ID:", txid)
print("Date:", date)
print("From:", sender)
print("To:", receiver)
print("Type:", category)
print("Amount:", amount, "BTC")
print("Wallet fee:", transaction_fee, "BTC")

records.append({
    "Transaction_ID": txid,
    "Date": date,
    "From": sender,
    "To": receiver,
    "Type": category,
    "Amount_BTC": str(amount),
    "Wallet_Fee_BTC": str(transaction_fee)
})
```

```
    # Net change calculated from the supplied records.
    net_change = incoming_total - outgoing_total - fee_total

    return (
        records,
        incoming_total,
        outgoing_total,
        fee_total,
        net_change,
        transaction_count
    )


def generate_report(records, filename):
    """Export transaction details to a CSV file."""

    fields = [
        "Transaction_ID",
        "Date",
        "From",
        "To",
        "Type",
        "Amount_BTC",
```

```
        "Wallet_Fee_BTC"
    ]

    try:
        with open(filename, "w", newline="",
                  encoding="utf-8") as file:

            writer = csv.DictWriter(file, fieldnames=fields)
            writer.writeheader()
            writer.writerows(records)

        print("\nTransaction report saved:", filename)

    except OSError as error:
        print("Could not save report:", error)


def display_summary(records, incoming, outgoing, fees,
                    net_change, transaction_count):
    """Display transaction statistics."""

    print("\n" + "=" * 65)
    print("                  FORENSIC SUMMARY")
    print("=" * 65)
```

```
    print("Valid records analyzed:", len(records))
    print("Incoming total:", incoming, "BTC")
    print("Outgoing total:", outgoing, "BTC")
    print("Wallet-paid fees:", fees, "BTC")
    print("Net balance change:", net_change, "BTC")

    print("\nTransaction classification:")
    for category, count in transaction_count.items():
        print(category, ":", count)

    print("\nAnalysis completed at:",
          datetime.now().strftime("%Y-%m-%d %H:%M:%S"))

    print("=" * 65)


def main():
    print("CRYPTOCURRENCY WALLET FORENSICS")
    print("------------------------------")

    filename = input(
        "Enter wallet data filename (wallet_data.json): "
    ).strip()
```

```
if not filename:
    filename = "wallet_data.json"

if not os.path.isfile(filename):
    print("Error: File does not exist:", filename)
    return

data = load_wallet_data(filename)

if data is None:
    return

results = analyze_transactions(data)

records, incoming, outgoing, fees, net_change, counts = results

generate_report(records, "transaction_report.csv")

display_summary(
    records, incoming, outgoing, fees, net_change, counts
)
```

```
if __name__ == "__main__":
    main()
```